1. Write a program to rotate the following shape
   a. Rotate line AB whose endpoints are A (2, 5) and B (6, 12) about the origin through a 30° clockwise direction.
   b. Rotate line AB whose endpoints are input by the user about origin through a 30° anti-clockwise direction.
2. A triangle ABC is given. The coordinates of A, B, and C are given as A (3 4),B (6 4),C (4 8) Write a program to reflect the position of the triangle i.e., to the x-axis.
3. Write a program to perform the following
   a. Transformation, rotation, scaling
4. Write a program to perform Line Clipping using the Sutherland Algorithm user can input line coordinates.
5. Program to clip a line using the Liang Barsky Method
6. Program to implement Standard Perspective Projection in 3-Dimensions.
7. Program to implement Parallel Projection in 3-Dimensions.
8. C program to demonstrate drawing a circle using OpenGL

Program to rotate a line:

```
#include<stdio.h>
#include<graphics.h>
#include<math.h>
int main()
{
   intgd=0,gm,x1,y1,x2,y2;
   double s,c, angle;
   initgraph(&gd, &gm, "C:\\TC\\BGI");
   setcolor(RED);
   printf("Enter coordinates of line: ");
   scanf("%d%d%d%d",&x1,&y1,&x2,&y2);
   cleardevice();
   setbkcolor(WHITE);
   line(x1,y1,x2,y2);
   getch();
   setbkcolor(BLACK);
   printf("Enter rotation angle: ");
   scanf("%lf", &angle);
   setbkcolor(WHITE);
   c = cos(angle *3.14/180);
   s = sin(angle *3.14/180);
   x1 = floor(x1 * c + y1 * s);
   y1 = floor(-x1 * s + y1 * c);
   x2 = floor(x2 * c + y2 * s);
   y2 = floor(-x2 * s + y2 * c);
   cleardevice();
   line(x1, y1 ,x2, y2);
```

```
   getch();
   closegraph();
return 0;
}
```

Program to rotate a Triangle:

```
#include<stdio.h>
#include<graphics.h>
#include<math.h>
main()
{
   intgd=0,gm,x1,y1,x2,y2,x3,y3;
   double s,c, angle;
   initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");
   setcolor(RED);
   printf("Enter coordinates of triangle: ");
   scanf("%d%d%d%d%d%d",&x1,&y1,&x2,&y2, &x3, &y3);
   setbkcolor(WHITE);
   cleardevice();
   line(x1,y1,x2,y2);
   line(x2,y2, x3,y3);
   line(x3, y3, x1, y1);
   getch();
   setbkcolor(BLACK);
   printf("Enter rotation angle: ");
   scanf("%lf", &angle);
   setbkcolor(WHITE);
   c = cos(angle *M_PI/180);
   s = sin(angle *M_PI/180);
```

```
   x1 = floor(x1 * c + y1 * s);
   y1 = floor(-x1 * s + y1 * c);
   x2 = floor(x2 * c + y2 * s);
   y2 = floor(-x2 * s + y2 * c);
   x3 = floor(x3 * c + y3 * s);
   y3 = floor(-x3 * s + y3 * c);
   cleardevice();
   line(x1, y1 ,x2, y2);
   line(x2,y2, x3,y3);
   line(x3, y3, x1, y1);
   getch();
   closegraph();
   return 0;
}
```

Program to perform Mirror Reflection about a line:

```
#include <iostream.h>
#include <conio.h>
#include <graphics.h>
#include <math.h>
#include <stdlib.h>
#define pi 3.14
class arc
{
   float x[10],y[10],theta,ref[10][10],ang;
         float p[10][10],p1[10][10],x1[10],y1[10],xm,ym;
   int i,k,j,n;
   public:
   void get();
```

```
    void cal ();
    void map ();
    void graph ();
    void plot ();
    void plot1();
};
void arc::get ()
{
    cout<<"\n ENTER ANGLE OF LINE INCLINATION AND Y INTERCEPT";
    cin>> ang >> b;
    cout <<"\n ENTER NO OF VERTICES";
    cin >> n;
    cout <<"\n ENTER";
    for (i=0; i<n; i++)
    {
        cout<<"\n x["<<i<<"] and y["<<i<<"]";
    }
    theta =(ang * pi)/ 180;
    ref [0] [0] = cos (2 * theta);
    ref [0] [1] = sin (2 * theta);
    ref [0] [2] = -b *sin (2 * theta);
    ref [1] [0] = sin (2 * theta);
    ref [1] [1] = -cos (2 * theta);
    ref [1] [2] = b * (cos (2 * theta)+1);
    ref [2] [0]=0;
    ref [2] [1]=0;
    ref [2] [2] = 1;
}
void arc :: cal ()
```

```
{
   for (i=0; i < n; i++)
   {
      p[0] [i] = x [i];
      p [1] [i] = y [i];
      p [2] [i] = 1;
   }
   for (i=0; i<3;i++)
   {
      for (j=0; j<n; j++)
      {
         p1 [i] [j]=0;
         for (k=0;k<3; k++)
      }
      p1 [i] [j] + = ref [i] [k] * p [k] [j];
          }
for (i=0; i<n; i++)
  {
   x1 [i]=p1[0] [i];
   y1 [i] = p1 [1] [i];
   }
}
void arc :: map ()
{
   int gd = DETECT,gm;
   initgraph (&gd, &gm, " ");
         int errorcode = graphresult ();
   /* an error occurred */
   if (errorcode ! = grOK)
```

```
    {
        printf ("Graphics error: %s \n", grapherrormsg (errorcode));
        printf ("Press any key to halt:");
        getch ();
        exit (1); /* terminate with an error code */
    }
}
void arc :: graph ()
{
    xm=getmaxx ()/2;
    ym=getmaxy ()/2;
    line (xm, 0, xmm 2*ym);
}
void arc :: plot 1 ()
{
    for (i=0; i <n-1; i++)
    {
        circle (x1[i]+xm, (-y1[i]+ym), 2);
        line (x1[i]+xm, (-y1[i]+ym), x1[i+1]+xm, (-y1[i+1]+ym));
    }
        line (x1[n-1)+xm, (-y1[n-1]+ym), x1[0]+xm, (-y1[0]+ym));
        getch();
}
void arc :: plot ()
{
    for (i=0; i <n-1; i++)
    {
        circle (x1[i]+xm, (-y1[i]+ym, 2);
        line (x1[i]+xm, (-y1[i]+ym), x[i+1]+xm, (-y1[i+1]+ym));
```

```
    }
        line (x[n-1]+xm, (-y1[n-1]+ym), x[0]+xm, (-y[0]+ym));
        getch();
}
void main ()
{
    class arc a;
    clrscr();
    a.map();
    a.graph();
    a.get();
    a.cal();
    a.plot();
    a.plot1();
    getch();
}
```

Write a program of Translation, Rotation, and Scaling using Composite Transformation.

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<math.h>
int x1,y1,x2,y2,x3,y3,a,b;
void draw();
void rotate();
int main(void)
{
int gd=DETECT,gm;
initgraph(&gd,&gm,"C:\\TC\\BGI");
printf("Enter first co-ordinate value for triangle:");
```

```
scanf("%d%d",&x1,&y1);
printf("Enter second co-ordinatevalues for triangle:");
scanf("%d%d",&x2,&y2);
printf("Enter third co-ordinate valuesfor triangle:");
scanf("%d%d",&x3,&y3);
draw();
getch();
rotate();
getch();

return 0;
}

void draw()
{
  line(x1,y1,x2,y2);
  line(x2,y2,x3,y3);
  line(x3,y3,x1,y1);
}
 void rotate()
 {
    int a1,a2,a3,b1,b2,b3;
    float angle;
    printf("Enter the rotation angle co-ordinates:");
    scanf("%f",&angle);
    cleardevice();
      angle=(angle*3.14)/180;
      a1=a+(x1-a)*cos(angle)-(y1-b)*sin(angle);
      b1=b+(x1-a)*sin(angle)+(y2-b)*cos(angle);
```

```
    a2=a+(x2-a)*cos(angle)-(y1-b)*sin(angle);
    b2=b+(x2-a)*sin(angle)+(y2-b)*cos(angle);
    a3=a+(x3-a)*cos(angle)-(y1-b)*sin(angle);
    b3=b+(x3-a)*sin(angle)+(y2-b)*cos(angle);
    printf("ROTATION");
    printf("\n Changed coordinates\n");
    printf("%d %d\n%d %d\n%d %d",a1,b1,a2,b2,a3,b3);
  line(a1,b1,a2,b2);
  line(a2,b2,a3,b3);
  line(a3,b3,a1,b1);
}
```

Write a program to draw a Circle using midpoint implementation Method.

```
#include<stdio.h>
#include<graphics.h>

void drawcircle(int x0, int y0, int radius)
{
  int x = radius;
  int y = 0;
  int err = 0;

  while (x >= y)
  {
  putpixel(x0 + x, y0 + y, 7);
  putpixel(x0 + y, y0 + x, 7);
  putpixel(x0 - y, y0 + x, 7);
  putpixel(x0 - x, y0 + y, 7);
```

```
    putpixel(x0 - x, y0 - y, 7);
    putpixel(x0 - y, y0 - x, 7);
    putpixel(x0 + y, y0 - x, 7);
    putpixel(x0 + x, y0 - y, 7);
          if (err <= 0)
    {
       y += 1;
       err += 2*y + 1;
    }

    if (err > 0)
    {
       x -= 1;
       err -= 2*x + 1;
    }
    }
}
 void main()
{
    int gdriver=DETECT, gmode, error, x, y, r;
    initgraph(&gdriver, &gmode, "c:\\turboc3\\bgi");
          printf("Enter radius of circle: ");
    scanf("%d", &r);

    printf("Enter co-ordinates of center(x and y): ");
    scanf("%d%d", &x, &y);
    drawcircle(x, y, r);
    getch();
}
```

Program to clip a line using Liang Barsky Method

```
#include<iostream.h>

#include <stdio.h>

#include<conio.h>

#include<graphics.h>

#include<process.h>

class liang

{

   float x1,x2,y1,y2,u1,u2,dx,dy,xm,ym;

   float xmin, ymin, xmax, ymax,p[4],q[4],r[5],r[5];

   int gd, gm, test, s, k;

   public:

   void clipliang();

   void get();

   void map();

   void graph();

};

void liang :: map()

{

   gd=DETECT;

   initgraph (&gd, &gm, "");

   int errorcode = graphresult();

   /*an error occurred */

         if (errorcode!=grOK)

   {

      printf("Graphics error: %s \n",grapherrormsg (errorcode));

                  printf("Press and key to halt: ");

                  getch();
```

```
                exit(1); /* terminate with an error code */
        }
}
void liang::graph()
{
   xm=getmaxx()/2;
   ym=getmaxy()/2;
   line (xm, 0, xm, 2 * ym);
   line (0, ym, 2 * xm, ym);
}
void liang :: get()
{
   cout<<"\n ENTER WINDOW COORDINATES xwmin, ywmin, xwmax, ywmax";
   cin>>xmin>>ymin>>xmax>>ymax>>;
   rectangle (xmin+xm,-ymin+ym,xmax+xm, -ymax+ym);
     //  rectangle (320+xwmin, 240-ywmax, 320+xwmax, 240-ywmin);
        cout<<"\n ENTER END POINTS OF LINE (x1, y1)(x2, y2) ";
        cin>>x1>>y1>>x2>>y2;
   line (x1+xm,-y1+ym, x2+xm,-y2+ym);
   getch();
}
     // line (x1, y1, x2, y2);
void liang :: clipliang()
{
   float x=0, y=1;
   dx=x2-x1;
   dy=y2-y1;
   p[0]=-dx;
   p[1]=dx;
```

```
    p[2]=-dy;
    p[3]=dy;
    q[0]=x1-xmin;
    q[1]=xmax-x1;
    q[2]=y1-ymin;
    q[3]=ymax-y1;
    for (k=0;k<4;k++)
    {
       if (p[k]==0&&q[k]<0)
       {
          cout<<"\n OUTSIDE";
                        getch();
                        exit(0);
                  }
                   if (p[k]<0)
             {
          r[k]=q[k]/p[k];
          if (r1[k]<y)
             y=r1[k];
                u2=y;
       }
    }
       if (u1>u2)
       {
          cout<<"\n COMPLETE OUTSIDE WINDOW";
                        getch();
                        exit(0);
                    }
void main ()
```

```
{
   liang1;
   clrscr();
   l.map();
   l.graph();
   l.get();
   l.clipliang();
   getch();
}
```

Program to implement Standard Perspective Projection in 3-Dimensions.

```
#include<iostream.h>
#include<conio.h>
#include<graphics.h>
#include<math.h>
#include<stdio.h>
#include<stdlib.h>
#define pi 3.14
class per
{
   float x[10], y[10], z [10], d;
   float x1 [10], y1[10], xm, ym, z1 [10];
   int i, k, j, n;
   public:
   void get();
   void perpro();
   void map();
   void graph();
   void plot1();
}
```

```
void per::get()
{
   cout<< "\n ENTER OF VERTICES";
         cin>>n;
   cout<<"\n ENTER";
         for (i=0;i<n;i++)
   {
      cout<<"\n x["?i<<"], y["<<i<<"] and z ["<<i<<"]";
      cin>>x[i]>>y[i]>>z[i];
   }
      cout<<"\n ENTER WIDTH FOR Z-AXIS-d";
      cin>>d;
}
void per:: perpro()
{
   for(i=0;i<n;i++)
   {
      x1[i]=(d*x[i])/(z[i]+d);
      y1[i]=(d*y[i])/(z[i]+d);
      z1[i]=0;
   }
   cout<< "\n ENTER PROJECTION, VERTICES OF POLYGON ARE--";
         for(i=0;i<n;i++)
   {
      cout<<"x?["<<i<<"]="<<x1[i]<< "\t";
      cout<<"y?["<<i<<"]="<<y1[i]<< "\t";
      cout<<"z?["<<i<<"]="<<z1[i]<< "\t";
      cout<<"\n";
   }
```

```
}
void per :: map()
{
   int gd=DETECT;
   initgraph (&gd, &gm, "");
   int errorcode = graphresult();
   /*an error occurred */
        if (errorcode!=grOK)
   {
      printf("Graphics error: %s \n",grapherrormsg (errorcode));
                printf("Press and key to halt: ");
                getch();
                exit(1); /* terminate with an error code */
         }
}
void per::graph()
{
   xm= get maxx()/2;
   ym=get maxy()/2
   line (xm,0, xm, 2 * ym);
   line (0, ym, 2 * xm, ym);
}
void per::plot1()
{
   for(i=0;i<n-1;i++)
   {
      line(x1[i]*100+xm,(-y1[i]*100+ym),x1[i+1]*100+xm,(-y1[i+1]*100+ym));
   }
line(x1[n-1]*100+xm,(-y1[n-1]*100+ym),x1[0]*100+xm,(-y1[0]*100+ym));
```

```
   getch();
}
void main()
{
   class per a;
   clrscr();
   a.map();
   a.graph();
   a.get();
   a.perpro();
   a.graph();
   a.plot1();
   getch();
}
```

Program to implement Parallel Projection in 3-Dimensions.

```
#include<iostream.h>
#include<conio.h>
#include<graphics.h>
#include<math.h>
#include<stdio.h>
#include<stdlib.h>
#define pi 3.14
class per
{
   float x[10], y[10], z [10], d, m[10], q1[10], r1[10], m1[10];
   float x1 [10], y1[10], xm, ym, z1 [10], cav[4] [4], p[10] [10];
   float p1[10] [10];
   float a, b, c, ang, theta, f;
```

```
    int i, k, j, n;
    public:
    void get();
    void par_v();
    void par_vnr0();
    void map();
    void graph();
    void project(float m[],float q[], float r[]);
};
void per:: perproject(float m[], float q[], float r[])
{
    cout<<"\n ENTER WIDTH FOR Z-AXIS d";
    cin>>d;
    cout<< "\n AFTER PROJECTION, VERTICES OF POLYGON ARE--";
          for(i=0;i<n;i++)
    {
       cout<<"x?["<<i<<"]="<<x1[i]<< "\t";
       cout<<"y?["<<i<<"]="<<y1[i]<< "\t";
       cout<<"z?["<<i<<"]="<<z1[i]<< "\t";
       cout<<"\n";
    }
    getch();
    clearviewport();
    graph();
    for(i=0;i<n-1;i++)
    {
       line(m[i]*100+xm,(-q[i]*100+ym),m[i+1]*100+xm,(-n1[i+1]*100+ym));


          }
```

```
   line(m[n-1]*100+xm,(-q[n-1]*100+ym),m[0]*100+xm,(-q[0]*100+ym));
getch();
}
void per::get()
{
   cout<< "\n ENTER OF VERTICES";
         cin>>n;
   cout<<"\n ENTER";
         for (i=0;i<n;i++)
   {
      cout<<"\n x["?i<<"], y["<<i<<"] and z ["<<i<<"]";
      cin>>x[i]>>y[i]>>z[i];
   }
      cout<<"\n ENTER WIDTH FOR Z-AXIS-d";
      cin>>d;
      int ch=100;
      do
      {
         cout<< "\n 1->orthographic projection";
                        cout<<"\n 2->oblique projection";
                        cout<<"\n 0-> exit";
         cout<<"\n enter ch";
         cin>>ch;
         switch (ch)
         {
            case1:
               int ch1=100;
            do
            {
```

```
        cout<<"\n 1->for xy viewplane";
        cout<<"\n 2->for yz viewplane";
        cout"\n 3->for zx viewplane";
        cout<<"\n 0->exit";
        cout<<"\n enter ch1";
        cin>>ch1;
        switch(ch1)
        {
          case1:
            x1[i]=x[i];
            y1[i]=y[i];
            z1[i]=0;
            project(x1,y1,z1);
            break;
          case2:
            x1[i]=0;
            y1[i]=0;
            z1[i]=z[i];
            project(x1,y1,z1);
            break;
          case3:
            x1[i]=x[i];
            y1[i]=0;
            z1[i]=z[i];
            project(x1,y1,z1);
            break;
        }
    }
while (ch!=0)
```

```
            break;
              case2:
          ch1=100;
          do
          {
             cout<<"\n 1->on xy plane";
             cout<<"\n 2->cavalier";
             cout<<"\n 0->exit";
             cout<<"\n enter ch1";
             cin>>ch1;
             switch(ch1)
             {
                case1:
                   par_v();
                   break;
                case2:
                   par_vnr0();
                   break();
             }
             while(ch!=0);
             break;
          }
          while(ch!=0)
       }
void per::par_v()
{
   cout<<"\n enter v (a, b, c) " ;
   cin>>a>>b>>c;
   for(i=0; i<n;i++)
```

```
{
   x1[i]=x[i]-((a*z[i]/c);
   y1[i]=y[i]-((b*z[i]/c);
   project(x1,y1, z1);
}
 }
void per::par_vnr0()
{
cout<<"\n enter f";
cin>>f;
cout<<"\n enter angle";
cin>>angle;
theta= ((pi/180)* ang);
for(i=0;i<4;i++)
{
   for(j=0;j<4;j++)
   {
      if(i==j)
      cav[i] [j]=1;
      else
         cav[i] [j]=0;
   }
}
cav[0] [2]=f*cos(theta);
cav[1][2]=f*sin (theta);
cav[2][2]=0;
for(i=0;i<n;i++)
{
   p[0][i]=x[i];
```

```
        p[1][i]=y[i];
        p[2][i]=z[i];
        p[3][i]=1;
    }
    for(i=0;i<4;i++)
    {
        for(j=0;j<n;j++)
        {
            p1[i][j]=0;
            for(k=0;k<4;k++)
            {
                p1[i][j]+=cav[i][k]*p[k][j];
            }
        }
    }
    for(i=0;i<n;i++)
    {
        x1[i]=p1[0][i];
        y1[i]=p1[1][i];
        z1[i]=p1[i];
    }
    project(x1,y1,z1);
    }
void per :: map()
{
  int gd=DETECT;
  initgraph (&gd, &gm, "");
  int errorcode = graphresult();
  /*an error occurred */
```

```
        if (errorcode!=grOK)
   {
      printf("Graphics error: %s \n",grapherrormsg (errorcode));
                printf("Press and key to halt: ");
                getch();
                exit(1); /* terminate with an error code */
         }
}
void per::graph()
{
   xm= get maxx()/2;
   ym=get maxy()/2
   line (xm,0, xm, 2 * ym);
   line (0, ym, 2 * xm, ym);
}
void main()
{
   class per a;
   clrscr();
   a.map();
   a.get();
   getch();
}
```